Graphify: evaluación de incorporación

› Grafo de conocimiento determinista para ELECTRONICA — guía para el HP EliteDesk 805 G6

Autor: Prof. CERO

Fecha: 11 de octubre de 2026

Índice

1 Resumen Ejecutivo

Se evaluó **Graphify** (`Graphify-Labs/graphify`, versión **v0.9.85**, publicada el 10 de octubre de 2026) como herramienta de representación y consulta del código del workspace `ELECTRONICA`. Graphify convierte un directorio de código, documentación, esquemas SQL, configuraciones y PDFs en un **grafo de conocimiento consultable** (`graph.json`) construido de forma **determinista** a partir de árboles sintácticos (*tree-sitter*), **sin LLM y sin base de datos vectorial**.

El veredicto es **adoptable como herramienta de solo-código, tras un piloto medido**. Su principio de diseño —un script determinista que produce un artefacto reproducible— es exactamente el que sostiene la arquitectura de 3 capas del proyecto. La migración a la **HP EliteDesk 805 G6** (16 GB de RAM) elimina además el único obstáculo de recursos que hoy lo bloquea en el equipo Celeron (3,63 GB).

Veredicto rápido: SÍ, adoptable tras un piloto

Graphify aporta una capa que el proyecto **hoy no tiene**: las relaciones `calls`/`imports`/`inherits` resueltas *entre ficheros* como un grafo tipado y verificable. **No sustituye** al RAG actual (`rag_system.py`): son complementarios. La única regla no negociable es que su *pase semántico* (docs/PDF/imágenes, que sí usa un LLM) no debe romper el determinismo del proyecto.

Recuerda

Guardarraíl determinista. Adoptar Graphify como *herramienta externa* no viola las 3 capas: no introduce lógica de decisión en el chat ni un flujo nuevo. Sí lo violaría (a) resolver su pase semántico en prosa de chat, o (b) crear un `flujo_*` que lo use sin su directiva y su script. Cualquier integración real exige las tres capas: `directives/*.yaml` + orquestador + `execution/*.py`.

2 Qué es Graphify

Graphify es una *skill* para asistentes de IA respaldada por una biblioteca Python. La *skill* orquesta la biblioteca; la biblioteca puede usarse sola. El paquete de PyPI se llama `graphifyy` (doble “y”), mientras que el comando de consola es `graphify`.

Ficha de la herramienta

Repositorio	`Graphify-Labs/graphify`
Versión evaluada	v0.9.85 (10 de octubre de 2026)
Licencia	Apache-2.0 + MIT
Lenguaje	Python ($\geq$ 3.10)
Paquete PyPI	`graphifyy` (comando: `graphify`)
Dependencias base	`networkx`, `numpy`, `rapidfuzz` y $\sim$26 gramáticas `tree-sitter`
Salida	`graph.json`, `graph.html`, `GRAPH_REPORT.md`
Vectores	**Ninguno** (no hay *embeddings*)
Telemetría	Ninguna (registra solo consultas en `~/.cache/`)

2.1 El pipeline determinista

Toda la construcción del grafo para **código** es local y sin LLM:

$$\texttt{detect()} \rightarrow \texttt{extract()} \rightarrow \texttt{build()} \rightarrow \texttt{cluster()} \rightarrow \texttt{report} \rightarrow \texttt{export}$$

Cada etapa vive en su propio módulo y se comunican por diccionarios y grafos `networkx`. El código se analiza con **árboles sintácticos** (tree-sitter, unos 40 lenguajes), **sin enviar nada fuera de la máquina**. Solo el pase semántico sobre documentación, PDFs, imágenes y vídeo requiere un *backend* LLM, y únicamente si se le configura uno.

Concepto clave

Cada arista está etiquetada. Una conexión se marca `EXTRACTED` si es explícita en el código fuente (un `import`, una llamada directa) e `INFERRED` si la dedujo el resolver. Las ambiguas se marcan `AMBIGUOUS` y se aíslan para revisión humana. Esto es lo que separa a Graphify de una caja negra: se puede auditar qué se leyó y qué se dedujo.

2.2 Artefactos que produce

```
graphify-out/
|-- graph.html          (grafo interactivo: abrir en el navegador)
|-- GRAPH_REPORT.md     (god nodes, comunidades, preguntas sugeridas)
`-- graph.json          (el grafo completo, consultable sin releer
    ficheros)
```

El grafo incluye los “god nodes” (los conceptos más conectados), las comunidades detectadas por el algoritmo de Leiden y rutas entre símbolos (`path`) o explicaciones de un concepto (`explain`).

Aviso de deriva (*churn*)

Cadencia de publicación muy alta. Entre el 6 y el 10 de octubre de 2026 se publicaron las versiones 0.9.78 a 0.9.85 (una por día, aproximadamente). El ritmo es una señal de salud del proyecto, pero también de **deriva de esquema y de API**: por eso el grafo lleva `schema_version` y `graphify_version`, y conviene fijar una versión concreta en lugar de seguir `latest`.

3 Encaje con la arquitectura de 3 capas

Principio rector compartido

Electrónica separa orquestación probabilística de ejecución determinista. **Graphify** hace lo mismo: un script determinista (`extract`, tree-sitter) produce un artefacto estable; el LLM solo lo consume. No son dos filosofías distintas, es la misma aplicada al código como grafo.

- **Determinismo.** Los últimos ocho lanzamientos son, casi en su totalidad, correcciones de determinismo y portabilidad: los identificadores de nodo ya no filtran la ruta de *checkout* ni el usuario del sistema operativo, de modo que **un clon produce el mismo grafo**. Es la misma garantía "mismo input → mismo output" del enrutador.
- **Integración con OpenCode.** Graphify soporta explícitamente `-platform opencode`; el *plugin* admite el contrato v1 (`server()`) y v2 (`setup()`). Encaja con el motor rotativo del asistente y, en modo solo-código, **no consume** la clave de OpenRouter.
- **Servidor MCP opcional.** El extra `graphifyy[mcp]` expone un servidor (`graphify-mcp`) que se suma a los servidores MCP del proyecto. Es un proceso aparte, por lo que no colisiona con el pin de `mcp` < 2 del resto.
- **Seguridad.** Todas las entradas externas pasan por `graphify/security.py`: validación de URL (solo http/https, con bloqueo de redirecciones `file://`), límite de tamaño y tiempo, y rutas de grafo que deben resolverse *dentro* de `graphify-out/`.

4 Beneficios concretos

- **Consultar en vez de *grep*.** `graphify query`, `graphify path` y `graphify explain` sustituyen la lectura fichero a fichero, lo que reduce los tokens que consume el orquestador.
- **Relaciones entre ficheros.** Resuelve `calls`, `imports`, `inherits` y `mixes_in` entre módulos, algo que el RAG actual **no** hace.
- **Verificable.** Las etiquetas `EXTRACTED`/`INFERRED` y la resolución *fail-closed* ante ambigüedad hacen el resultado auditable.

- **Racional y referencias.** Los comentarios `# NOTE:` / `# WHY:` y las citas a ADR/RFC se vuelven nodos de primera clase enlazados al código.
- **Coste cero en solo-código.** `extract -code-only` corre completamente offline y no gasta créditos.

5 Riesgos y costes

Riesgo	Detalle	Severidad
Recursos	En el equipo Celeron había **641 MB** de RAM disponibles y el repo pesa **1,4 GB** (2105 ficheros). Con la HP EliteDesk (16 GB) el riesgo **baja a residual** para solo-código; el paso semántico de PDFs/imágenes sigue siendo el tramo pesado.	Media → Baja
Dependencias	`uv tool install` crea un entorno aislado (no toca `elect_env`), pero descarga ∼26 gramáticas de tree-sitter; el extra `leiden` arrastra `graspologic` (scipy/sklearn/numba) en Python < 3.13.	Media
Edita el árbol de trabajo	Escribe `graphify-out/` en el directorio actual y, con `graphify hook install`, **reconstruye en cada commit**.	Media
Edita `AGENTS.md`	En OpenCode Graphify es plataforma de fichero de instrucciones: `install` inserta guía en `AGENTS.md`, que en este repo es **la fuente de verdad**. Un merge automático puede dañarla.	**Alta**
Créditos / guardarraíl	El pase semántico manda docs/PDFs a un *backend* LLM. Usarlo *inline* violaría el guardarraíl. Debe ir por `execution/enrutador.py` o quedarse en `-code-only`.	Media
Deriva (*churn*)	Cadencia casi diaria y marcado enfoque a la nube. Conviene fijar una versión.	Media
Disco de 256 GB	El SSD del equipo elegido es modesto (ya al 91 % en el equipo actual); mantener `chroma_db`, `datasets` y modelos fuera del disco de sistema.	Media

6 Comparación con el RAG actual

Aspecto	Graphify	rag_system.py (ChromaDB)
Unidad	Grafo tipado: nodos = conceptos, aristas explicadas	*Embeddings* y similitud
Determinismo	Sí (para código)	No (vectores aproximados)
Almacén vectorial	**No**	Sí (`chroma_db/`)
Cubre código	Sí (unos 40 lenguajes)	No (docs, `.tex`, `.pdf`)
Coste por construcción	0 créditos en `--code-only`	LLM por consulta
Relación	Estructura ("qué llama a qué")	Contenido ("qué dice qué")

Concepto clave

No es un sustituto, es una capa complementaria. Graphify aporta la *estructura* del código; el RAG aporta el *contenido* de la documentación. Conviven sin solaparse: una pregunta "¿qué llama a `calcular_nota`?" es de Graphify; "¿qué dice la rúbrica sobre el montaje?" es del RAG.

7 Veredicto

Recomendación

Adoptar Graphify como herramienta de solo-código, en tres fases y sin tocar todavía la arquitectura del repo. El cuello de botella real del equipo Celeron era la RAM y el volumen, no el software —el mismo patrón que ya se documentó para el MCP de FreeCAD—. La migración a la HP EliteDesk resuelve la RAM; el volumen del repo y el disco de 256 GB siguen siendo el punto a medir.

- **Fase 0 — Alcance.** Decidir si el piloto cubre solo los lenguajes vivos (`execution/`, `.opencode/`, los `flujo_*.py`) o el repo completo.
- **Fase 1 — Piloto aislado.** Instalar, construir el grafo y medir, sin modificar ningún fichero del repo.
- **Fase 2 — Instalación por proyecto** (`--project`), revisando y **mergeando a mano** el diff de `AGENTS.md`.
- **Fase 3 — Integración real** (si se quiere ir más allá): nueva pareja `directives/graphify_code.yaml` + `execution/graphify_build.py`.

8 Guía de implementación en la HP EliteDesk 805 G6

Esta sección es la operativa: qué ejecutar, en qué orden, y cómo comprobar que salió bien.

8.1 Contexto de hardware

HP EliteDesk 805 G6 Mini (equipo destino)

Procesador	AMD Ryzen 5 PRO 4650GE (Zen 2, 6C/12T, 35 W)
Instrucciones SIMD	AVX2 + FMA3 (clave frente al Celeron, sin AVX)
Memoria	16 GB DDR4 (frente a 3,63 GB del equipo actual)
Almacenamiento	SSD 256 GB (ampliable: 1× M.2 2280 y bahía 2,5" SATA)
Sistema operativo	Windows 11 Pro → se instalará Linux

Recuerda

El cómputo pesado sigue en **Google Colab**. Esta máquina habilita lo que el proyecto permite en local: inferencia, prototipos, validación y orquestación. Graphify cae del lado de la *orquestación*, no del entrenamiento.

8.2 Prerrequisitos

Requisito	Mínimo	Comprobación / instalación
Python	3.10+	`python3 -version` (el `elect_env` es 3.10)
`uv` (recomendado)	cualquiera	`curl-LsSfhttps://astral.sh/uv/install.sh|sh`
`pipx` (alternativa)	cualquiera	`pip install pipx`

En esta máquina de trabajo ya hay `uvx` 0.11.11 y Python 3.10.20, así que la ruta de `uv` es la natural.

8.3 Instalación (entorno aislado)

```
# 1. Instalar el CLI en un entorno aislado (no toca el entorno
    elect_env)
uv tool install graphifyy

# 2. Registrar la skill en el asistente OpenCode
graphify install --platform opencode
```

⚠ No dejar que reescriba `AGENTS.md`

No ejecutar `graphify install` sin plataforma: por diseño apunta a Claude Code. La forma correcta aquí es `graphify install -platform opencode`. Y antes de aceptar cualquier escritura sobre `AGENTS.md`, **revisar el diff** y fusionarlo a mano; nunca dejar que lo sobreescriba.

Extras opcionales (instalar solo lo que se use):

```
uv tool install "graphifyy[leiden]"   # deteccion de comunidades (Leiden)
uv tool install "graphifyy[mcp]"      # servidor MCP (graphify-mcp)
uv tool install "graphifyy[pdf]"      # extraccion de PDF (pase semantico)
uv tool install "graphifyy[all]"      # todo lo anterior y mas
```

8.4 Primera construcción del grafo

Para arrancar sin gastar créditos, indexar **solo el código** (offline):

```
cd ~/ELECTRONICA
graphify extract --code-only .      # construye graphify-out/ sin LLM
```

Dentro del asistente, la skill ofrece el atajo:

```
/graphify .                         # construir el grafo de la carpeta actual
/graphify ./docs --update           # re-extraer solo los ficheros cambiados
/graphify . --no-viz                # omitir el HTML, solo informe + JSON
```

Consultas habituales una vez construido el grafo:

```
graphify query "que conecta el enrutador con el cliente LLM?"
graphify path "enrutador.py" "llm_client.py"
graphify explain "calcular_nota"
```

8.5 Verificación de determinismo

El valor del proyecto está en la reproducibilidad: **dos construcciones deben producir el mismo** `graph.json`.

```
graphify extract --code-only . && sha256sum graphify-out/graph.json > /tmp/g1.sha
graphify extract --code-only . && sha256sum graphify-out/graph.json > /tmp/g2.sha
diff /tmp/g1.sha /tmp/g2.sha && echo "DETERMINISTA: hashes identicos"
```

Si los hashes difieren, **detenerse y diagnosticar** antes de dar el piloto por bueno: un grafo no reproducible contradice el guardarraíl del proyecto.

8.6 Integración opcional

- **Servidor MCP.** Con el extra `[mcp]`, `graphify-mcp` expone el grafo como herramientas MCP, sumándose a los servidores ya existentes.
- **Hook de git.** `graphify hook install` reconstruye el grafo en cada commit y *checkout*. Útil, pero añade coste por commit: evaluar antes de dejarlo permanente.
- **Pase semántico.** Si en algún momento se indexan docs/PDFs, la llamada al LLM debe pasar por `execution/enrutador.py` (`-task destilacion`), nunca decidirse en el chat.
- **Control de versiones.** Decidir conscientemente si `graphify-out/` se ignora o se comparte; no dejarlo al azar.

8.7 Convivencia con la arquitectura de 3 capas

Reto

Si la adopción pasa de *herramienta* a *flujo*, hace falta su tríada completa: `directives/graphify_code.yaml` (el *qué*), un orquestador que valide entradas/salidas, y `execution/graphify_build.py` como capa 3 determinista (envoltorio que llama al CLI, valida el JSON y respeta el *retry budget* de 3 intentos). Nunca un orquestador sin directiva ni script.

9 Criterios de aceptación

El piloto se considera superado cuando se cumplan **todos** estos puntos:

- `uv tool install graphifyy` deja el comando `graphify` disponible sin romper `elect_env`.
- `graphify extract -code-only .` termina con código 0 y no consume créditos de OpenRouter.
- Dos construcciones consecutivas producen el **mismo** `graph.json` (hashes idénticos).
- `GRAPH_REPORT.md` identifica *god nodes* y comunidades coherentes con la estructura real del repo.

- Ningún fichero del repo queda modificado salvo, si se decide, el *merge* manual y revisado de `AGENTS.md`.
- Se anotan tiempos y consumo de RAM por construcción, para comparar el antes (Celeron) y el después (HP EliteDesk).

Recuerda

Al recibir la HP EliteDesk, antes de nada: confirmar la CPU (**4650GE**, no 4350GE), ejecutar `execution/env_diagnostic.py` (confirmar AVX2) y `flujo_auditar_sistema.py` (veredicto de RAM y servicios). Esos dos pasos ya están anotados como criterio de aceptación del documento de hardware.

10 Referencias

- **Repositorio:** https://github.com/Graphify-Labs/graphify — `README.md`, `ARCHITECTURE.md`, `SECURITY.md`, `pyproject.toml` y notas de versión v0.9.76–v0.9.85.
- **Documentación:** https://docs.graphify.com (guía de inicio, referencia de CLI, configuración, entradas soportadas).
- **Documento de hardware hermano:** `docs/PC_PARA_IA/HP_ELITEDESK_VEREDICTO.tex` — veredicto del equipo destino.
- **Sesión de continuidad:** `Sessions/2026-10-10_veredicto_hardware_hp_elitedesk.md` y `Sessions/2026-10-11_graphify_evaluacion_incorporacion.md`.